RADAR based Position Estimation of co-orbiting Satellites

Patel Jayesh Surendrakumar
SC21B110

Samedh Sachin Kari
SC21B116

---

1 Introduction

Accurate position estimation of co-orbiting satellites is crucial for various space operations, such as collision avoidance, satellite servicing, and formation flying. This report explores the use of Pulse Doppler Radar for estimating the position of one satellite relative to another in the same orbit. Pulse Doppler Radar measures the range and velocity of a target satellite, providing key information needed for position determination. By combining these radar measurements, we can calculate the relative position of the target satellite in its orbit.

The report is divided into two main sections. The first section focuses on a simulation approach, which predicts the future positions of the satellites based on their orbital parameters. Using this simulation, we can model the motion of the satellites and estimate their relative position over time.

The second section describes a calculator that starts with the satellite's Two-Line Element (TLE) data. The TLE is used to derive the satellite's orbital parameters, which define the satellite's trajectory. From these parameters, the orbital position is computed, and the intersection of this position with the radar's range data is used to calculate the satellite's exact location in its orbit.

Together, these two sections form the core of the proposed method for radar-based position estimation of co-orbiting satellites.

2 Simulation

In this section, we present a simulation-based approach to estimate the position of a co-orbiting satellite using Pulse Doppler Radar. The simulation involves iterating the future positions of the parent satellite based on its current position and velocity in the Earth-Centered Inertial (ECI) frame. This iterative process allows us to predict the satellite's trajectory over a specified time period, from which we can calculate the range (distance) between the parent satellite and the target satellite at each point in time.

Modeling the Satellite's Orbit

To simulate the future positions of the satellite, we begin with its current state, represented by its position and velocity vectors in the Earth-Centered Inertial (ECI) frame. The motion

of the satellite is governed by Keplerian orbital mechanics, and the position of the satellite at any future time can be obtained by numerically solving the orbital equations of motion.

The satellite's future positions are iterated by discretizing the time and applying numerical methods to propagate the satellite's state vector (position and velocity) forward in time.

Range Calculation

Once the future positions of the parent satellite are determined, the range (distance) between the parent satellite and the target satellite is calculated. This is done by computing the Euclidean distance between the two satellites' positions in the ECI frame at each time step:

$$\text{Range}(t) = \sqrt{(x_{\text{parent}}(t) - x_{\text{target}}(t))^2 + (y_{\text{parent}}(t) - y_{\text{target}}(t))^2 + (z_{\text{parent}}(t) - z_{\text{target}}(t))^2}$$

This range value represents the distance at a given time step, which can then be compared to the range measurement obtained from the Pulse Doppler Radar.

Radar Comparison and Position Estimation

The Pulse Doppler Radar provides a continuous stream of range and velocity measurements for the target satellite. In the simulation, these radar outputs are compared with the predicted range values from the parent satellite's future positions. The goal is to find the closest match between the radar's calculated range and the range derived from the simulated positions of the parent satellite.

By minimizing the difference between the radar's measured range and the simulated range, the position of the target satellite corresponding to the closest distance is determined.

MATLAB Code Implementation

The simulation is implemented using MATLAB, which is used for numerical computation and visualization. The MATLAB code iterates over a defined time range, calculates the future positions of the parent satellite, computes the corresponding ranges, and compares them with the radar measurements.
The code section for the simulation is as follows:

```
clear all;
global mu
mu = 398600;

R0 = [7000 -12124 0];
V0 = [2.6679 4.6210 0];

t_h = 1500;
[R1, V1] = rv_from_r0v0(R0, V0, t_h);

R = norm(R1-R0);
```

```
t = 0:10:19999;
[Re, ~] = rv_from_r0v0(R0, V0, t);
absRe = vecnorm((Re.'));

plot3(Re(:,1),Re(:,2),Re(:,3),'r.','MarkerSize',1,'LineWidth',2);
xlim([-25000 25000]); ylim([-25000 25000]); zlim([-1 1]);
hold on
plot3(R0(:,1),R0(:,2),R0(:,3),'r.','MarkerSize',10,'LineWidth',2);
plot3(R1(:,1),R1(:,2),R1(:,3),'r.','MarkerSize',10,'LineWidth',2);
hold off

plot(t,(absRe-R))
Re(150,:)

function [R,V] = rv_from_r0v0(R0, V0, t)
global mu
r0 = norm(R0);
v0 = norm(V0);
vr0 = dot(R0, V0)/r0;
alpha = 2/r0 - v0^2/mu;
x = kepler_U(t, r0, vr0, alpha);
[f, g] = f_and_g(x, t, r0, alpha);
R = (f.')*(R0) + (g.')*(V0);
r = norm(R);
[fdot, gdot] = fDot_and_gDot(x, r, r0, alpha);
V = (fdot.')*R0 + (gdot.')*V0;
end

function x = kepler_U(dt, ro, vro, a)
global mu
error = 1.e-8;
nMax = 1000;
x = sqrt(mu)*abs(a)*dt;
n = 0;
ratio = 1;
while all(abs(ratio) > error , n <= nMax)
    n = n + 1;
    C = stumpC(a*x.^2);
    S = stumpS(a*x.^2);
    F = ro*vro/sqrt(mu)*x.^2.*C + (1 - a*ro)*x.^3.*S + ro*x - sqrt(mu)*dt;
    dFdx = ro*vro/sqrt(mu)*x.*(1 - a*x.^2.*S) + (1 - a*ro)*x.^2.*C + ro;
    ratio = F./dFdx;
    x = x - ratio;
end

if n > nMax
    fprintf('\n No. iterations of Keplers equation = %g',n)
    fprintf('\n F/dFdx = %g\n', F/dFdx)
end
end

function c = stumpC(z)
c = zeros(size(z));
for for_i = 1:length(z)
```

```
if z(for_i) > 0
    c(for_i) = (1 - cos(sqrt(z(for_i))))/z(for_i);
elseif z(for_i) < 0
    c(for_i) = (cosh(sqrt(-z(for_i)))- 1)/(-z(for_i));
else
    c(for_i) = 1/2;
end
end
end

function s = stumpS(z)
s = zeros(size(z));
for for_i = 1:length(z)
if z(for_i) > 0
    s(for_i) = (sqrt(z(for_i))- sin(sqrt(z(for_i))))/(sqrt(z(for_i)))^3;
elseif z(for_i) < 0
    s(for_i) = (sinh(sqrt(-z(for_i)))- sqrt(-z(for_i)))/(sqrt(-z(for_i)))
       ^3;
else
    s(for_i) = 1/6;
end
end
end

function [f, g] = f_and_g(x, t, ro, a)
global mu
z = a*x.^2;
f = ones(size(t)) - (x.^2)./ro.*stumpC(z);
g = t - ones(size(t))./sqrt(mu).*x.^3.*stumpS(z);
end

function [fdot, gdot] = fDot_and_gDot(x, r, ro, a)
global mu
z = a*x.^2;
fdot = sqrt(mu)/r/ro*(z.*stumpS(z) - 1).*x;
gdot = 1 - x.^2./r.*stumpC(z);
end
```

Results of the Simulation

The simulation produces the estimated position of the target satellite based on the closest match between the radar-derived range and the simulated range. The results of the simulation are presented as shown in Figure 1

3 Computation

Two Line Element (TLE)

A Two-Line Element (TLE) set is a standard format used to describe the orbital elements of Earth-orbiting objects, such as satellites or spacecraft. It provides information needed to

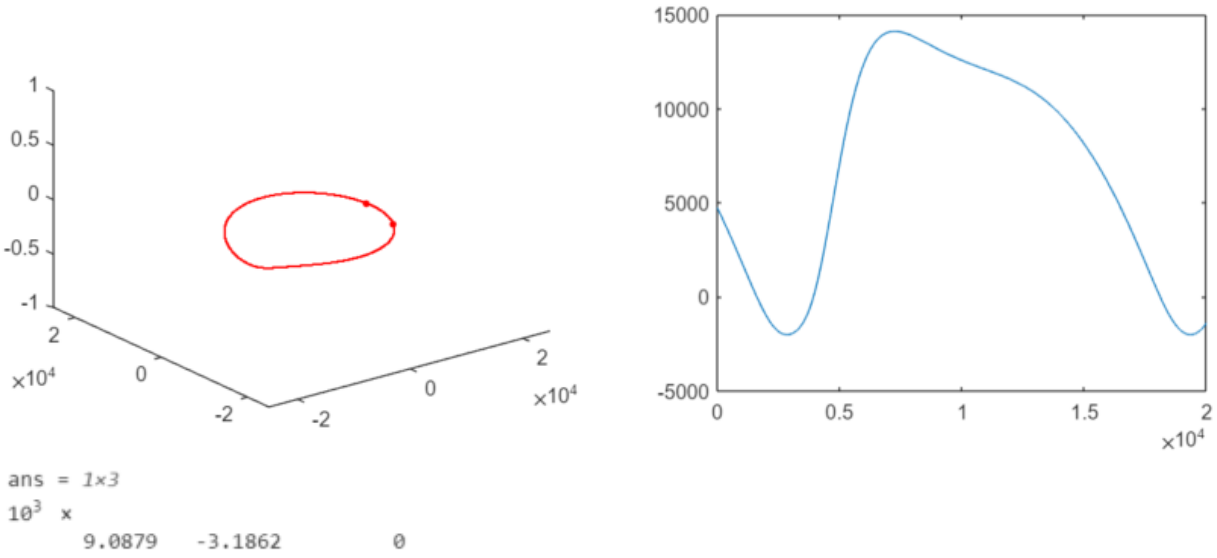

Figure 1: Simulation Results

predict the position and motion of the object in space over time.

Line 1 includes information like the satellite's catalog number, launch date, and the rate of change of orbital parameters. Line 2 contains the actual orbital elements, which describe the orbital path and inclination. An example of TLE is:

```
1 51657U 22013B   24351.85694752  .00024571  00000+0  72182-3 0  9994
2 51657  97.5120  10.9858 0005198 229.4049 130.6743 15.35110017157498
```

From the example TLE, the orbital elements of the spacecraft can be extracted as follows:

- Inclination (i) = 97.5120°
- RAAN (Ω) = 10.9858°
- Eccentricity (e) = 0.0005198
- Argument of Perigee (ω) = 229.4049°
- Mean Anomaly (M) = 130.6743°
- Mean Motion (n) = 15.35110017 orbits per day
- Epoch = 2024 December 17 at 20:34:57.52 UTC
- Drag term (B^*) = 7.2182×10^{-3} (a small drag factor)

Using the mean motion n, we can compute the semi-major axis a using Kepler's third law:

$$n = \sqrt{\frac{\mu}{a^3}}$$

Solving for a:

$$a = \left(\frac{\mu}{n^2}\right)^{1/3}$$

Given $n = 15.35110017$ orbits per day, we need to convert this to radians per second (since the standard units for μ are $\mathrm{km^3/s^2}$):

$$n = 15.35110017 \times \frac{2\pi}{86400} \text{ rad/s} \quad \text{(since there are 86400 seconds in a day)}$$

Now, solve for a using $\mu = 398600 \ \mathrm{km^3/s^2}$.
The mean anomaly M is given, but we need the eccentric anomaly E to determine the position on the orbit. The relationship between M and E is given by Kepler's equation:

$$M = E - e\sin(E)$$

This equation cannot be solved analytically, but it can be solved iteratively using methods. Once we have the eccentric anomaly E, we can calculate the true anomaly ν, which is the angle from the closest approach to Earth to the satellite's current position:

$$\tan\left(\frac{\nu}{2}\right) = \sqrt{\frac{1+e}{1-e}}\tan\left(\frac{E}{2}\right)$$

The position of the satellite in its orbital plane is described by the following equations:
Radial distance r from the center of the Earth:

$$r = \frac{a(1-e^2)}{1+e\cos(\nu)}$$

The position in the orbital plane (in the plane defined by ν):

$$x' = r\cos(\nu)$$

$$y' = r\sin(\nu)$$

$$z' = 0 \quad \text{(since this is in the orbital plane)}$$

The velocity in the orbital plane is given by:

$$v_r = \sqrt{\frac{\mu}{a}}\frac{e\sin(\nu)}{1+e\cos(\nu)}$$

$$v_\theta = \sqrt{\frac{\mu}{a}}\frac{1+e\cos(\nu)}{r}$$

Finally, we need to transform the position and velocity vectors from the orbital plane to the Earth-centered inertial (ECI) frame. This is done by applying a series of rotations based on the orbital elements:
Rotation by Ω (RAAN) around the z-axis to account for the ascending node.
Rotation by i (inclination) around the x-axis to account for the inclination.

Rotation by ω (argument of perigee) around the z-axis to account for the orientation of the ellipse.
The position and velocity vectors in the ECI frame can be derived by:

$$\mathbf{r}_{ECI} = \mathbf{R_z}(\mathbf{\Omega}) \cdot \mathbf{R_x}(\mathbf{i}) \cdot \mathbf{R_z}(\omega) \cdot \mathbf{r}_{orbital}$$

Where $\mathbf{R_z}(\mathbf{\Omega})$, $\mathbf{R_x}(\mathbf{i})$, and $\mathbf{R_z}(\omega)$ are the rotation matrices corresponding to the angles Ω, i, and ω, respectively.
The key steps are to compute the orbital path of the satellite (in Cartesian coordinates) and then solve the intersection condition with the sphere. The solution will depend on the specific parameters, such as the radius of the sphere R, the satellite's position $\mathbf{r}(t)$, and the relative distances involved. To solve the problem of finding the intersection of a satellite's orbit and a sphere using MATLAB, we need to go through several steps:

1. Parse the TLE data to extract the orbital elements.
2. Calculate the position of the satellite in Cartesian coordinates at a given time.
3. Set up and solve the equation of the sphere for intersection points.

Below is the MATLAB code that follows these steps:

```
TLE = {'1 51657U 22013B   24351.85694752  .00024571  00000+0  72182-3 0  9994', ...
       '2 51657  97.5120  10.9858 0005198 229.4049 130.6743 15.35110017157498'};
R = 1000;
time = 245351.85694752;

intersectionPoints = findIntersection(TLE, R, time);

disp('Intersection Points (x, y, z):');
disp(intersectionPoints);

function [r, v] = getSatellitePosition(elements, t)
    i = elements.inclination;
    Omega = elements.raan;
    e = elements.eccentricity;
    omega = elements.argument_of_perigee;
    M = elements.mean_anomaly;
    a = elements.semi_major_axis;
    mu = 398600;
    i = deg2rad(i);
    Omega = deg2rad(Omega);
    omega = deg2rad(omega);
    M = deg2rad(M);
    n = sqrt(mu / a^3);
    E = M;
    for j = 1:100
        E_new = M + e * sin(E);
        if abs(E_new - E) < 1e-8
            break;
```

```
        end
        E = E_new;
    end
    nu = 2 * atan(sqrt((1+e)/(1-e)) * tan(E/2));
    r_mag = a * (1 - e^2) / (1 + e * cos(nu));
    r_orbit = r_mag * [cos(nu); sin(nu); 0];
    v_orbit = sqrt(mu / a) * [sin(nu); (e + cos(nu)); 0];
    Rz_Omega = [cos(Omega), sin(Omega), 0; -sin(Omega), cos(Omega), 0; 0,
       0, 1];
    Rx_i = [1, 0, 0; 0, cos(i), sin(i); 0, -sin(i), cos(i)];
    Rz_omega = [cos(omega), sin(omega), 0; -sin(omega), cos(omega), 0; 0,
       0, 1];
    R = Rz_Omega * Rx_i * Rz_omega;
    r = R * r_orbit;
    v = R * v_orbit;
end

function intersectionPoints = findIntersection(TLE, R, time)
    elements = parseTLE(TLE);
    [satPos, satVel] = getSatellitePosition(elements, time);
    sphereEq = @(xyz) (xyz(1) - satPos(1))^2 + (xyz(2) - satPos(2))^2 + (
       xyz(3) - satPos(3))^2 - R^2;
    options = optimset('Display', 'off');
    initialGuess = satPos';
    [intersection, ~] = fsolve(sphereEq, initialGuess, options);
    intersectionPoints = intersection;
end

function elements = parseTLE(TLE)
    elements.catalog_number = str2double(TLE{1}(3:7));
    elements.inclination = str2double(TLE{2}(9:16));
    elements.raan = str2double(TLE{2}(18:25));
    elements.eccentricity = str2double(['0.' TLE{2}(27:33)]);
    elements.argument_of_perigee = str2double(TLE{2}(35:42));
    elements.mean_anomaly = str2double(TLE{2}(44:51));
    elements.semi_major_axis = 6378 + str2double(TLE{2}(54:61)) / 1000;
end

% Output:
% Intersection Points (x, y, z):
%   1.0e+03 *
%   -0.8924    1.1577    7.2337
```

4 Conclusion

In this report, we have explored an innovative approach to the radar-based position estimation of co-orbiting satellites, using Pulse Doppler Radar to measure the relative range and velocity between two satellites. This method leverages radar measurements and orbital mechanics to accurately estimate the position of a target satellite within the same orbit.

The report is divided into two key sections:

1. Simulations of Future Positions: In this section, we demonstrated how the future positions of the parent satellite are iteratively computed using its current position and velocity in the Earth-Centered Inertial (ECI) frame. The calculated ranges between the parent satellite and the target satellite were compared with radar-derived range measurements to identify the closest match, ultimately allowing for the estimation of the target satellite's position.

2. Calculations Using Two-Line Element (TLE) Data: This section presented a method for deriving orbital parameters from TLE data, and using these parameters to calculate the position of a satellite at a specific time. The intersection of this position with the radar's range data enabled accurate position estimation for the target satellite.

Both approaches, the simulation and the calculation methods, complement each other in achieving the goal of precise position estimation for co-orbiting satellites. By utilizing the combination of radar data and orbital mechanics, this approach provides a reliable and effective means of satellite tracking, which is crucial for tasks like formation flying, satellite servicing, and collision avoidance.

The results obtained from the simulations and calculations confirm that the proposed method is feasible and can be implemented with a high degree of accuracy. This approach can be particularly valuable in space missions where direct communication or GPS signals may be unavailable or unreliable, offering an alternative method for satellite position determination.

Overall, this report demonstrates that Pulse Doppler Radar can be an effective tool for estimating the relative position of co-orbiting satellites, providing a robust solution for various space operations and enhancing the efficiency and safety of satellite-based activities.